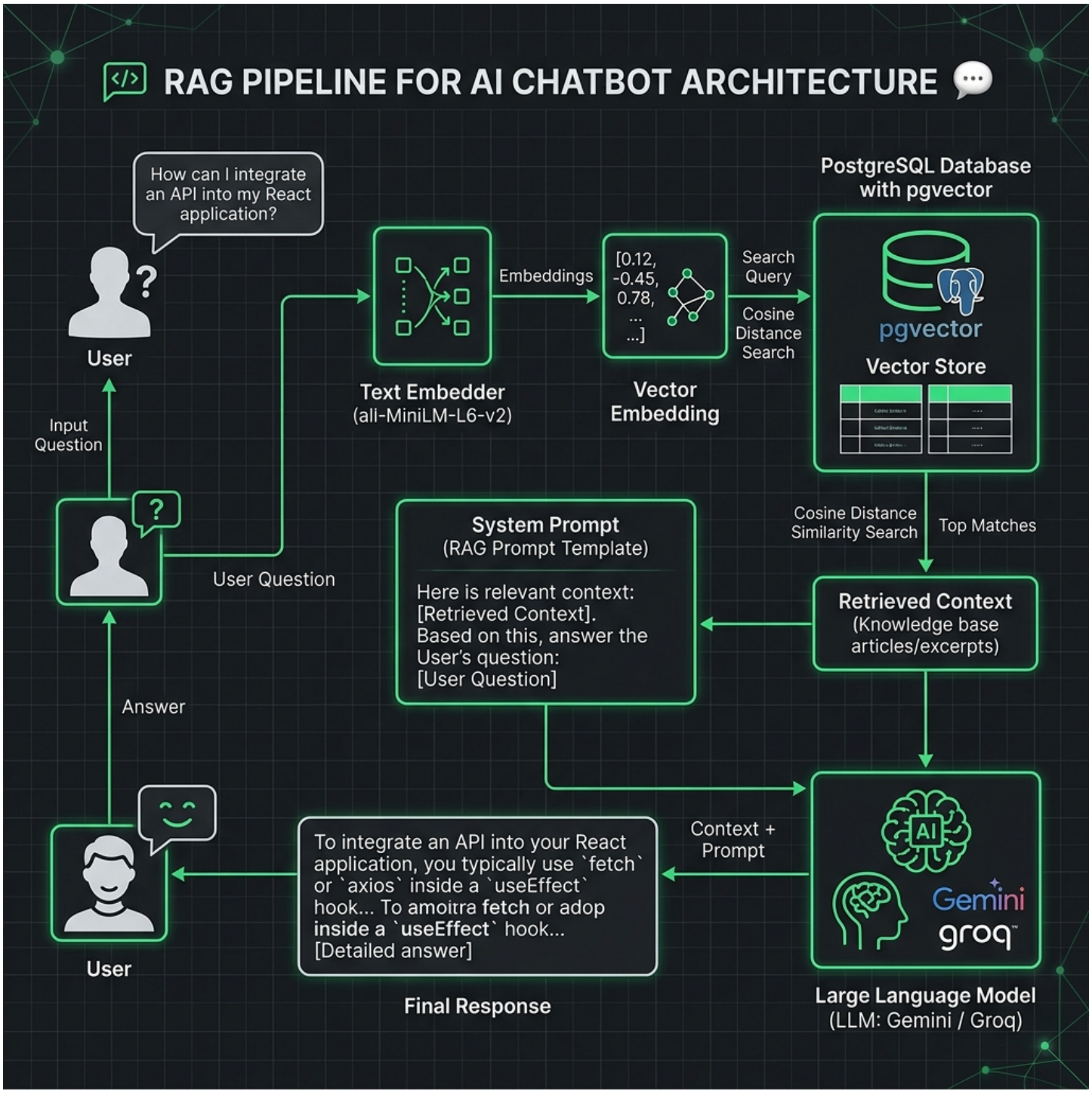
Varsapradaya FAQ Chatbot

RAG (Retrieval-Augmented Generation) Architecture

This document details the system design and architecture flow of the semantic search and Retrieval-Augmented Generation (RAG) system running inside the Varsapradaya AI Chatbot. The pipeline utilizes pgvector on PostgreSQL for high-speed, accurate, concept-based matching.



Varsapradaya FAQ Chatbot

RAG (Retrieval-Augmented Generation) Architecture

Step-by-Step Flow Explanation

1. User Query Entry

The user types a query in the frontend chatbox (e.g., 'What is Soil Sync?'). The message is POSTed asynchronously to the backend FastAPI endpoint (/api/chat) along with a unique thread_id.

2. State Machine Routing

The checkpointer loads the user's current conversation state from the database. Since the user is active, it dispatches the request to the 'chatting' node in the LangGraph graph configuration.

3. Embedding Generation

The chatting node routes the text query to a local SentenceTransformer model (all-MiniLM-L6-v2) inside a separate thread to prevent event-loop blockages. The model converts the text into a 384-dimensional floating-point semantic vector.

4. pgvector Cosine Search

A SQL query executes against the 'faq_embeddings' table using PostgreSQL pgvector. It computes the cosine distance (<=>) between the query vector and all stored FAQ vectors, filtering by a strict threshold of <= 0.76 to filter out off-topic questions.

5. Prompt Augmentation

The closest matching FAQ text (if it satisfies the distance check) is retrieved and formatted as plain text context. This context is injected directly into the LLM system prompt as 'MOST RELEVANT FAQ CONTEXT'.

6. Response Formulation

The augmented prompt and history are sent to the configured LLM provider (Groq/Gemini/OpenAI). The LLM is instructed to answer the user query relying only on the injected facts, ensuring 100% accuracy and preventing hallucinations.

7. Client Response

The FastAPI endpoint saves the assistant's reply to the message database table, and returns it to the client, which renders the answer bubble in the user interface.